

SQL Practice Exercises

Step1: Create Required Database and Tables

```
CREATE DATABASE EventManagementDB;
```

```
USE EventManagementDB;
```

#Create required Tables

#Users Table

```
CREATE TABLE Users (  
    user_id INT PRIMARY KEY AUTO_INCREMENT,  
    full_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    city VARCHAR(100) NOT NULL,  
    registration_date DATE NOT NULL  
);
```

#Events Table

```
CREATE TABLE Events (  
    event_id INT PRIMARY KEY AUTO_INCREMENT,  
    title VARCHAR(200) NOT NULL,  
    description TEXT,  
    city VARCHAR(100) NOT NULL,  
    start_date DATETIME NOT NULL,  
    end_date DATETIME NOT NULL,  
    status ENUM('upcoming','completed','cancelled'),  
    organizer_id INT,  
    FOREIGN KEY (organizer_id)  
        REFERENCES Users(user_id)  
);
```

#Sessions Table

```
CREATE TABLE Sessions (  
    session_id INT PRIMARY KEY AUTO_INCREMENT,  
    event_id INT,  
    title VARCHAR(200) NOT NULL,  
    speaker_name VARCHAR(100) NOT NULL,  
    start_time DATETIME NOT NULL,  
    end_time DATETIME NOT NULL,  
    FOREIGN KEY (event_id)  
        REFERENCES Events(event_id)  
);
```

#Registrations Table

```
CREATE TABLE Registrations (  
    registration_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    event_id INT,  
    registration_date DATE NOT NULL,  
    FOREIGN KEY (user_id)  
        REFERENCES Users(user_id),  
    FOREIGN KEY (event_id)  
        REFERENCES Events(event_id)  
);
```

#Feedback Table

```
CREATE TABLE Feedback (  
    feedback_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    event_id INT,
```

```

rating INT CHECK (rating BETWEEN 1 AND 5),
comments TEXT,
feedback_date DATE NOT NULL,
FOREIGN KEY (user_id)
    REFERENCES Users(user_id),
FOREIGN KEY (event_id)
    REFERENCES Events(event_id)
);

#Resources Table

CREATE TABLE Resources (
    resource_id INT PRIMARY KEY AUTO_INCREMENT,
    event_id INT,
    resource_type ENUM('pdf','image','link'),
    resource_url VARCHAR(255) NOT NULL,
    uploaded_at DATETIME NOT NULL,
    FOREIGN KEY (event_id)
        REFERENCES Events(event_id)
);

```

Step2: Insert the sample data into the created Tables

#Insert Sample Data

#Users

INSERT INTO Users

(user_id, full_name, email, city, registration_date)

VALUES

(1,'Alice Johnson','alice@example.com','New York','2024-12-01'),

(2,'Bob Smith','bob@example.com','Los Angeles','2024-12-05'),

(3,'Charlie Lee','charlie@example.com','Chicago','2024-12-10'),

```
(4,'Diana King','diana@example.com','New York','2025-01-15'),  
(5,'Ethan Hunt','ethan@example.com','Los Angeles','2025-02-01');
```

#Events

INSERT INTO Events

(event_id,title,description,city,start_date,end_date,status,organizer_id)

VALUES

```
(1,'Tech Innovators Meetup',  
'A meetup for tech enthusiasts.',  
'New York',  
'2025-06-10 10:00:00',  
'2025-06-10 16:00:00',  
'upcoming',  
1),
```

```
(2,'AI & ML Conference',  
'Conference on AI and ML advancements.',  
'Chicago',  
'2025-05-15 09:00:00',  
'2025-05-15 17:00:00',  
'completed',  
3),
```

```
(3,'Frontend Development Bootcamp',  
'Hands-on training on frontend tech.',  
'Los Angeles',  
'2025-07-01 10:00:00',  
'2025-07-03 16:00:00',  
'upcoming',  
2);
```

#Sessions

INSERT INTO Sessions

(session_id,event_id,title,speaker_name,start_time,end_time)

VALUES

(1,1,'Opening Keynote','Dr. Tech',

'2025-06-10 10:00:00',

'2025-06-10 11:00:00'),

(2,1,'Future of Web Dev','Alice Johnson',

'2025-06-10 11:15:00',

'2025-06-10 12:30:00'),

(3,2,'AI in Healthcare','Charlie Lee',

'2025-05-15 09:30:00',

'2025-05-15 11:00:00'),

(4,3,'Intro to HTML5','Bob Smith',

'2025-07-01 10:00:00',

'2025-07-01 12:00:00');

#Registrations

INSERT INTO Registrations

(registration_id,user_id,event_id,registration_date)

VALUES

(1,1,1,'2025-05-01'),

(2,2,1,'2025-05-02'),

(3,3,2,'2025-04-30'),

(4,4,2,'2025-04-28'),

(5,5,3,'2025-06-15');

#Feedback

INSERT INTO Feedback

(feedback_id,user_id,event_id,rating,comments,feedback_date)

VALUES

(1,3,2,4,'Great insights!','2025-05-16'),

(2,4,2,5,'Very informative.','2025-05-16'),

(3,2,1,3,'Could be better.','2025-06-11');

#Resources

INSERT INTO Resources

(resource_id,event_id,resource_type,resource_url,uploaded_at)

VALUES

(1,1,'pdf',

'https://portal.com/resources/tech_meetup_agenda.pdf',

'2025-05-01 10:00:00'),

(2,2,'image',

'https://portal.com/resources/ai_poster.jpg',

'2025-04-20 09:00:00'),

(3,3,'link',

'https://portal.com/resources/html5_docs',

'2025-06-25 15:00:00');

Step3:Verify the Created Tables

SELECT * FROM Users;

SELECT * FROM Events;

SELECT * FROM Sessions;

SELECT * FROM Registrations;

```
SELECT * FROM Resources;
```

[illegible]

Result Grid |   Filter Rows: | Edit:    | Export/Import:   | Wrap Cell Content: 

	registration_id	user_id	event_id	registration_date
▶	1	1	1	2025-05-01
	2	2	1	2025-05-02
	3	3	2	2025-04-30
	4	4	2	2025-04-29

Registrations

feedback_id	user_id	event_id	rating	comments	feedback_date
1	3	2	4	Great insights!	2025-05-16
2	4	2	5	Very informative.	2025-05-16
3	2	1	3	Could be better.	2025-06-11
NULL	NULL	NULL	NULL	NULL	NULL

Feedback

Result Grid					
Filter Rows:					
Edit:					
Export/Import:					
Wrap Cell Content:					
	resource_id	event_id	resource_type	resource_url	uploaded_at
▶	1	1	pdf	https://portal.com/resources/tech_meetup_age...	2025-05-01 10:00:00
	2	2	image	https://portal.com/resources/ai_poster.jpg	2025-04-20 09:00:00
	3	3	link	https://portal.com/resources/html5_docs	2025-06-25 15:00:00
*	NULL	NULL	NULL	NULL	NULL

Resources 6 x

Exercise 1: User Upcoming Events

Question

Show a list of all upcoming events a user is registered for in their city, sorted by date.

```

SELECT
    u.full_name,
    u.city,
    e.title AS event_name,
    e.start_date
FROM Users u
JOIN Registrations r
    ON u.user_id = r.user_id
JOIN Events e
    ON r.event_id = e.event_id
WHERE
    u.city = e.city
    AND e.status = 'upcoming'
ORDER BY
    e.start_date;

```

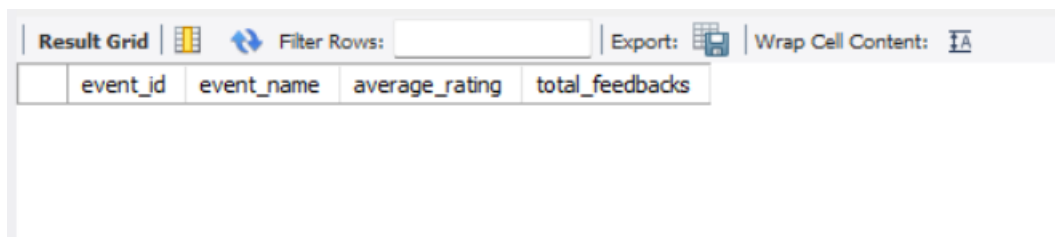
Result Grid				
Filter Rows:				
Export:				
Wrap Cell Content:				
	full_name	city	event_name	start_date
▶	Alice Johnson	New York	Tech Innovators Meetup	2025-06-10 10:00:00
	Ethan Hunt	Los Angeles	Frontend Development Bootcamp	2025-07-01 10:00:00

Exercise 2: Top Rated Events

Question

Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions.

```
SELECT
    e.event_id,
    e.title AS event_name,
    AVG(f.rating) AS average_rating,
    COUNT(f.feedback_id) AS total_feedbacks
FROM Events e
JOIN Feedback f
    ON e.event_id = f.event_id
GROUP BY
    e.event_id,
    e.title
HAVING
    COUNT(f.feedback_id) >= 10
ORDER BY
    average_rating DESC;
```



The screenshot shows a database query result grid with the following columns: event_id, event_name, average_rating, and total_feedbacks. The grid is empty, indicating that the query has not yet been executed or that there are no results to display.

event_id	event_name	average_rating	total_feedbacks
----------	------------	----------------	-----------------

Exercise 3: Inactive Users

Question

Retrieve users who have not registered for any events in the last 90 days.

```
SELECT
    u.user_id,
    u.full_name,
    u.email,
```

```

    u.city
FROM Users u
LEFT JOIN Registrations r
    ON u.user_id = r.user_id
    AND r.registration_date >= CURDATE() - INTERVAL 90 DAY
WHERE
    r.registration_id IS NULL;

```

Result Grid				
Filter Rows:				
Export:  Wrap Cell Content: 				
	user_id	full_name	email	city
▶	1	Alice Johnson	alice@example.com	New York
	2	Bob Smith	bob@example.com	Los Angeles
	3	Charlie Lee	charlie@example.com	Chicago
	4	Diana King	diana@example.com	New York
	5	Ethan Hunt	ethan@example.com	Los Angeles

Result 3 x 

Exercise 4: Peak Session Hours

Question

Count how many sessions are scheduled between 10 AM to 12 PM for each event.

```

SELECT
    e.event_id,
    e.title AS event_name,
    COUNT(s.session_id) AS session_count
FROM Events e
JOIN Sessions s
    ON e.event_id = s.event_id
WHERE
    TIME(s.start_time) >= '10:00:00'
    AND TIME(s.end_time) <= '12:00:00'
GROUP BY
    e.event_id,
    e.title;

```

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	event_id	event_name	session_count			
▶	1	Tech Innovators Meetup	1			
	3	Frontend Development Bootcamp	1			

Exercise 5: Most Active Cities

Question

List the top 5 cities with the highest number of distinct user registrations.

```
SELECT
    u.city,
    COUNT(DISTINCT r.user_id) AS total_registered_users
FROM Users u
JOIN Registrations r
    ON u.user_id = r.user_id
GROUP BY
    u.city
ORDER BY
    total_registered_users DESC
LIMIT 5;
```

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	city	total_registered_users				
▶	Los Angeles	2				
	New York	2				
	Chicago	1				

Exercise 6: Event Resource Summary

Question

Generate a report showing the number of resources (PDFs, images, links) uploaded for each event

```
SELECT
    e.event_id,
```

```

    e.title AS event_name,
    COUNT(r.resource_id) AS total_resources
FROM Events e
LEFT JOIN Resources r
    ON e.event_id = r.event_id
GROUP BY
    e.event_id,
    e.title
ORDER BY
    e.event_id;

```

Result Grid			
		Filter Rows:	
		Export:	
		Wrap Cell Content:	
	event_id	event_name	total_resources
▶	1	Tech Innovators Meetup	1
	2	AI & ML Conference	1
	3	Frontend Development Bootcamp	1

Exercise 7: Low Feedback Alerts

Question

List all users who gave feedback with a rating less than 3, along with their comments and associated event names.

```

SELECT
    u.user_id,
    u.full_name,
    e.title AS event_name,
    f.rating,
    f.comments
FROM Users u
JOIN Feedback f
    ON u.user_id = f.user_id
JOIN Events e
    ON f.event_id = e.event_id

```

WHERE

f.rating < 3;



Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	user_id	full_name	event_name	rating	comments
--	---------	-----------	------------	--------	----------

Result 7 x

Exercise 8: Sessions per Upcoming Event

Question

Display all upcoming events with the count of sessions scheduled for them.

SELECT

e.event_id,

e.title AS event_name,

COUNT(s.session_id) AS total_sessions

FROM Events e

LEFT JOIN Sessions s

ON e.event_id = s.event_id

WHERE

e.status = 'upcoming'

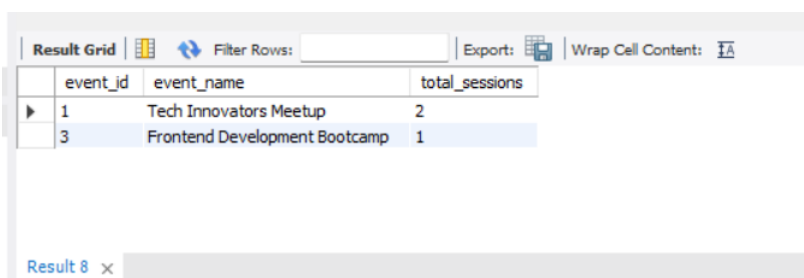
GROUP BY

e.event_id,

e.title

ORDER BY

e.event_id;



Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	event_id	event_name	total_sessions
▶	1	Tech Innovators Meetup	2
	3	Frontend Development Bootcamp	1

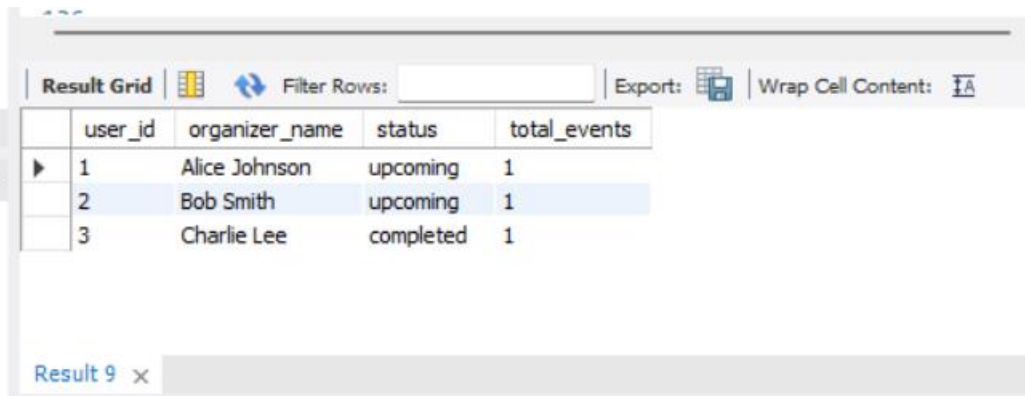
Result 8 x

Exercise 9: Organizer Event Summary

Question

For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).

```
SELECT
    u.user_id,
    u.full_name AS organizer_name,
    e.status,
    COUNT(e.event_id) AS total_events
FROM Users u
JOIN Events e
    ON u.user_id = e.organizer_id
GROUP BY
    u.user_id,
    u.full_name,
    e.status
ORDER BY
    u.user_id,
    e.status;
```



The screenshot shows a database query result grid with the following data:

	user_id	organizer_name	status	total_events
▶	1	Alice Johnson	upcoming	1
	2	Bob Smith	upcoming	1
	3	Charlie Lee	completed	1

The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' button. The result is labeled 'Result 9'.

Exercise 10: Feedback Gap

Question

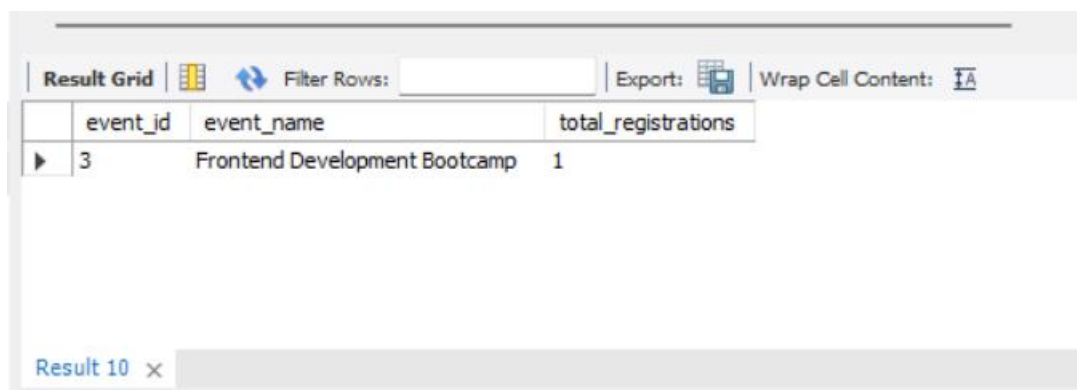
Identify events that had registrations but received no feedback at all.

```
SELECT
    e.event_id,
```

```

    e.title AS event_name,
    COUNT(r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r
    ON e.event_id = r.event_id
LEFT JOIN Feedback f
    ON e.event_id = f.event_id
WHERE
    f.feedback_id IS NULL
GROUP BY
    e.event_id,
    e.title;

```



The screenshot shows a database query result grid with the following columns: event_id, event_name, and total_registrations. The first row of data shows event_id 3, event_name 'Frontend Development Bootcamp', and total_registrations 1. The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' button. The bottom of the window shows 'Result 10' with a close button.

event_id	event_name	total_registrations
3	Frontend Development Bootcamp	1

Exercise 11: Daily New User Count

Question

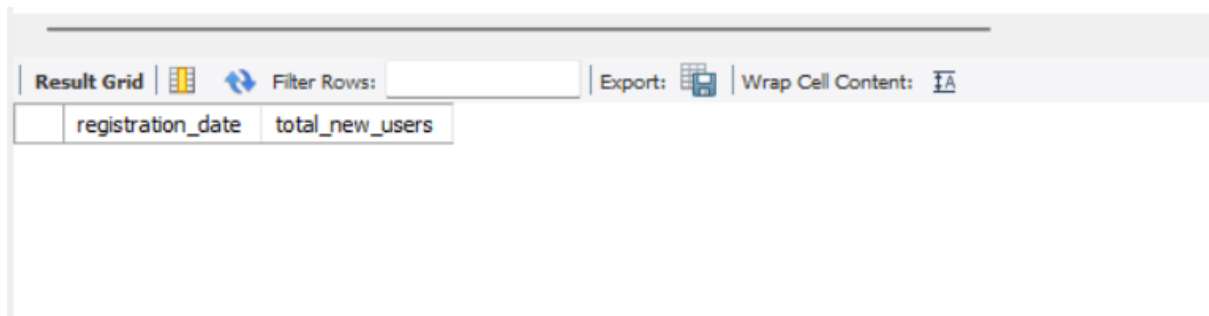
Find the number of users who registered each day in the last 7 days.

```

SELECT
    registration_date,
    COUNT(user_id) AS total_new_users
FROM Users
WHERE
    registration_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY
    registration_date
ORDER BY

```

registration_date;



The screenshot shows a database interface with a toolbar at the top containing 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with two columns: 'registration_date' and 'total_new_users'.

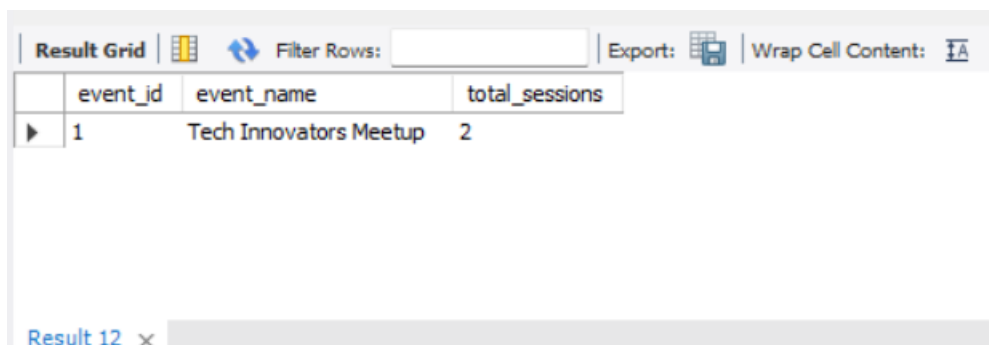
registration_date	total_new_users
-------------------	-----------------

Exercise 12: Event with Maximum Sessions

Question

List the event(s) with the highest number of sessions.

```
SELECT
    e.event_id,
    e.title,
    COUNT(s.session_id) AS total_sessions
FROM Events e
JOIN Sessions s
    ON e.event_id = s.event_id
GROUP BY
    e.event_id,
    e.title
ORDER BY
    total_sessions DESC
LIMIT 1;
```



The screenshot shows a database interface with a toolbar at the top containing 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with three columns: 'event_id', 'event_name', and 'total_sessions'. The first row shows event_id 1, event_name 'Tech Innovators Meetup', and total_sessions 2.

event_id	event_name	total_sessions
1	Tech Innovators Meetup	2

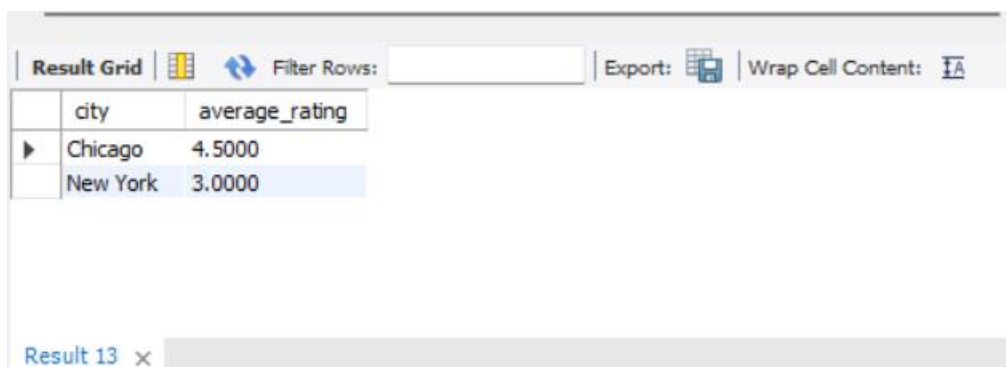
Result 12 x

Exercise 13: Average Rating per City

Question

Calculate the average feedback rating of events conducted in each city.

```
SELECT
    e.city,
    AVG(f.rating) AS average_rating
FROM Events e
JOIN Feedback f
    ON e.event_id = f.event_id
GROUP BY
    e.city
ORDER BY
    average_rating DESC;
```



The screenshot shows a database query result grid. The grid has two columns: 'city' and 'average_rating'. The results are sorted in descending order of average rating. Chicago has an average rating of 4.5000, and New York has an average rating of 3.0000. The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, and buttons for 'Export' and 'Wrap Cell Content'.

city	average_rating
Chicago	4.5000
New York	3.0000

Exercise 14: Most Registered Events

Question

List the top 3 events based on the total number of user registrations.

```
SELECT
    e.event_id,
    e.title,
    COUNT(r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r
    ON e.event_id = r.event_id
GROUP BY
```

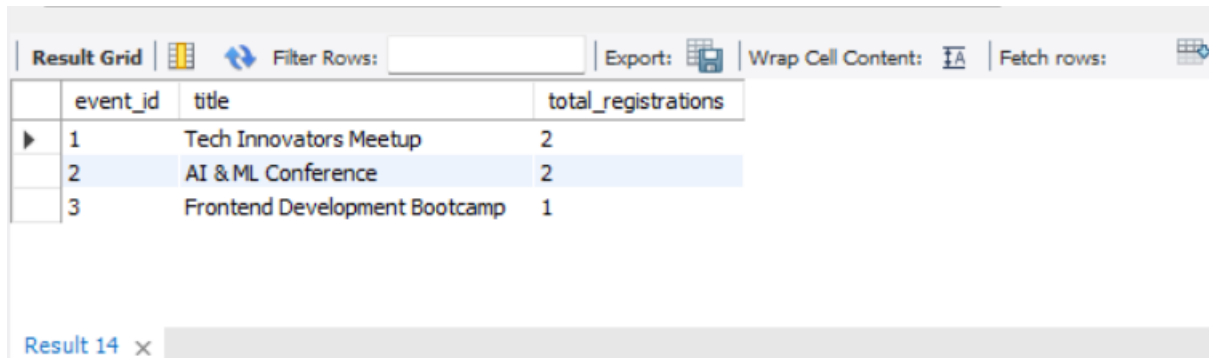
e.event_id,

e.title

ORDER BY

total_registrations DESC

LIMIT 3;



The screenshot shows a database query result grid with the following columns: event_id, title, and total_registrations. The results are ordered by total_registrations in descending order, limited to 3 rows.

event_id	title	total_registrations
1	Tech Innovators Meetup	2
2	AI & ML Conference	2
3	Frontend Development Bootcamp	1

Exercise 15: Event Session Time Conflict

Question

Identify overlapping sessions within the same event (i.e., session start and end times that conflict).

SELECT

s1.event_id,

s1.title AS session1,

s2.title AS session2

FROM Sessions s1

JOIN Sessions s2

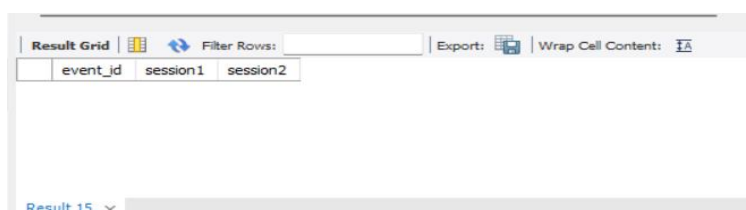
ON s1.event_id = s2.event_id

AND s1.session_id < s2.session_id

WHERE

s1.start_time < s2.end_time

AND s1.end_time > s2.start_time;



The screenshot shows a database query result grid with the following columns: event_id, session1, and session2. The results are currently empty.

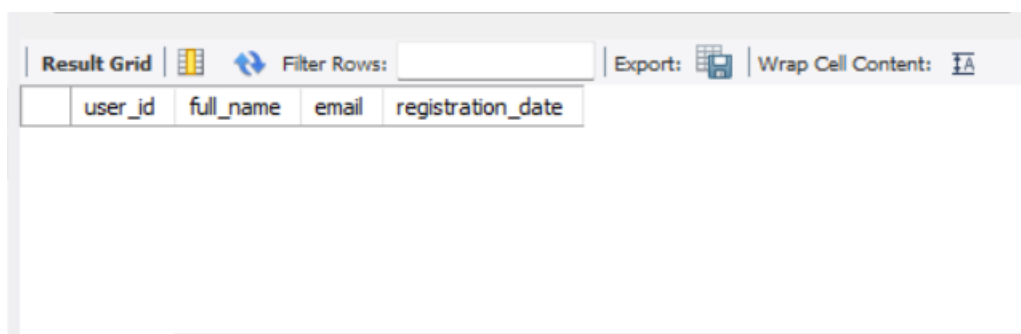
event_id	session1	session2
----------	----------	----------

Exercise 16: Unregistered Active Users

Question

Find users who created an account in the last 30 days but haven't registered for any events.

```
SELECT
    u.user_id,
    u.full_name,
    u.email,
    u.registration_date
FROM Users u
LEFT JOIN Registrations r
ON u.user_id = r.user_id
WHERE
    u.registration_date >= CURDATE() - INTERVAL 30 DAY
    AND r.registration_id IS NULL;
```



The screenshot shows a database query result grid. The top toolbar includes a 'Result Grid' button, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' button. Below the toolbar, the first row of the result set is visible, with columns labeled 'user_id', 'full_name', 'email', and 'registration_date'. The grid is currently empty of data rows.

Exercise 17: Multi-Session Speakers

Question

Identify speakers who are handling more than one session across all events.

```
SELECT
    speaker_name,
    COUNT(session_id) AS total_sessions
FROM Sessions
GROUP BY
    speaker_name
HAVING
```

Result Grid		 Filter Rows:	Export: 	Wrap Cell Content: 
	speaker_name	total_sessions		

Question

```
SELECT
    e.event_id,
    e.title
FROM Events e
LEFT JOIN Resources r
ON e.event_id = r.event_id
WHERE r.resource_id IS NULL;
```

[illegible]

Question

```
SELECT
    e.event_id,
    e.title,
```

```

COUNT(DISTINCT r.registration_id) AS total_registrations,
AVG(f.rating) AS average_rating
FROM Events e
JOIN Registrations r
ON e.event_id = r.event_id
JOIN Feedback f
ON e.event_id = f.event_id
WHERE e.status = 'completed'
GROUP BY
    e.event_id,
    e.title;

```

Result Grid				
		Filter Rows:		
		Export:		
		Wrap Cell Content:		
	event_id	title	total_registrations	average_rating
▶	2	AI & ML Conference	2	4.5000

Result 19 x

Exercise 20: User Engagement Index

Question

For each user, calculate how many events they attended and how many feedbacks they submitted.

```

SELECT
    u.user_id,
    u.full_name,
    COUNT(DISTINCT r.event_id) AS events_attended,
    COUNT(DISTINCT f.feedback_id) AS feedbacks_submitted
FROM Users u
LEFT JOIN Registrations r
ON u.user_id = r.user_id
LEFT JOIN Feedback f

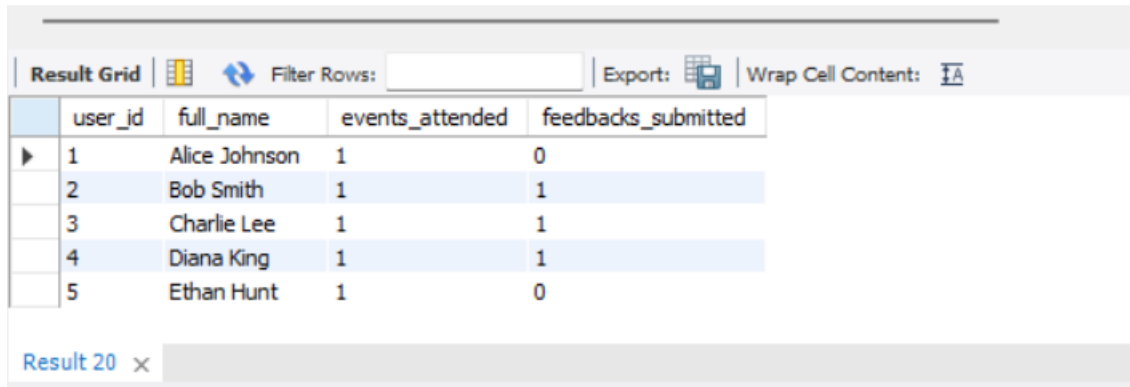
```

ON u.user_id = f.user_id

GROUP BY

u.user_id,

u.full_name;



The screenshot shows a database query result grid with the following columns: user_id, full_name, events_attended, and feedbacks_submitted. The data is as follows:

	user_id	full_name	events_attended	feedbacks_submitted
▶	1	Alice Johnson	1	0
	2	Bob Smith	1	1
	3	Charlie Lee	1	1
	4	Diana King	1	1
	5	Ethan Hunt	1	0

Below the table, it says "Result 20" with a close button (x).

Exercise 21: Top Feedback Providers

Question

List top 5 users who have submitted the most feedback entries.

SELECT

u.user_id,

u.full_name,

COUNT(f.feedback_id) AS total_feedbacks

FROM Users u

JOIN Feedback f

ON u.user_id = f.user_id

GROUP BY

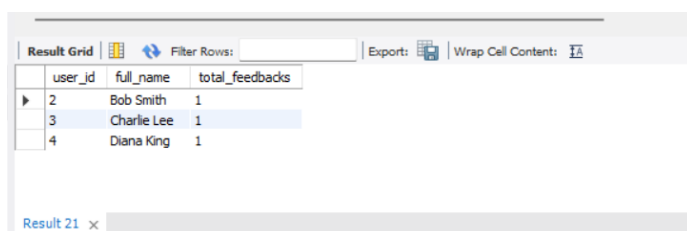
u.user_id,

u.full_name

ORDER BY

total_feedbacks DESC

LIMIT 5;



The screenshot shows a database query result grid with the following columns: user_id, full_name, and total_feedbacks. The data is as follows:

	user_id	full_name	total_feedbacks
▶	2	Bob Smith	1
	3	Charlie Lee	1
	4	Diana King	1

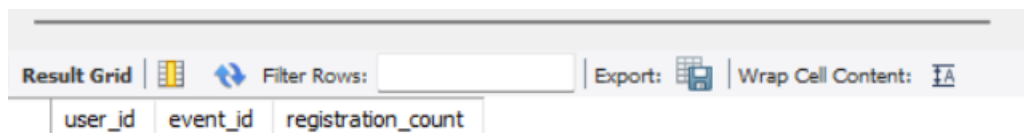
Below the table, it says "Result 21" with a close button (x).

Exercise 22: Duplicate Registrations Check

Question

Detect if a user has been registered more than once for the same event.

```
SELECT
    user_id,
    event_id,
    COUNT(*) AS registration_count
FROM Registrations
GROUP BY
    user_id,
    event_id
HAVING
    COUNT(*) > 1;
```



The screenshot shows a database query result grid. The top bar contains the text 'Result Grid' followed by icons for a grid, a refresh button, and a filter icon. To the right of the filter icon is a text input field labeled 'Filter Rows:'. Further right are icons for 'Export' and 'Wrap Cell Content:'. Below this bar, the first row of the result grid is visible, showing three columns: 'user_id', 'event_id', and 'registration_count'.

user_id	event_id	registration_count
---------	----------	--------------------

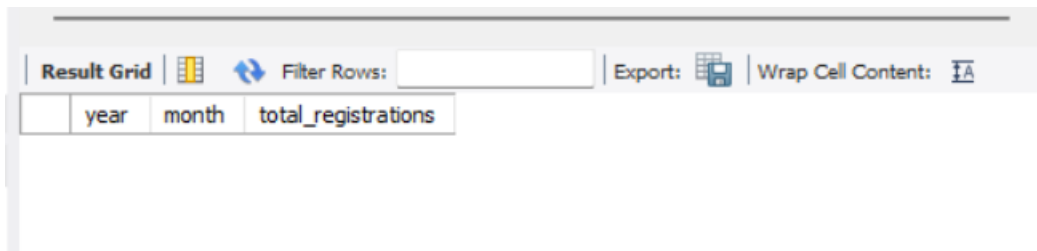
Exercise 23: Registration Trends

Question

Show a month-wise registration count trend over the past 12 months.

```
SELECT
    YEAR(registration_date) AS year,
    MONTH(registration_date) AS month,
    COUNT(*) AS total_registrations
FROM Registrations
WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH
GROUP BY
    YEAR(registration_date),
    MONTH(registration_date)
ORDER BY
    year,
```

month;



The screenshot shows a database interface with a toolbar at the top containing 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with three columns: 'year', 'month', and 'total_registrations'.

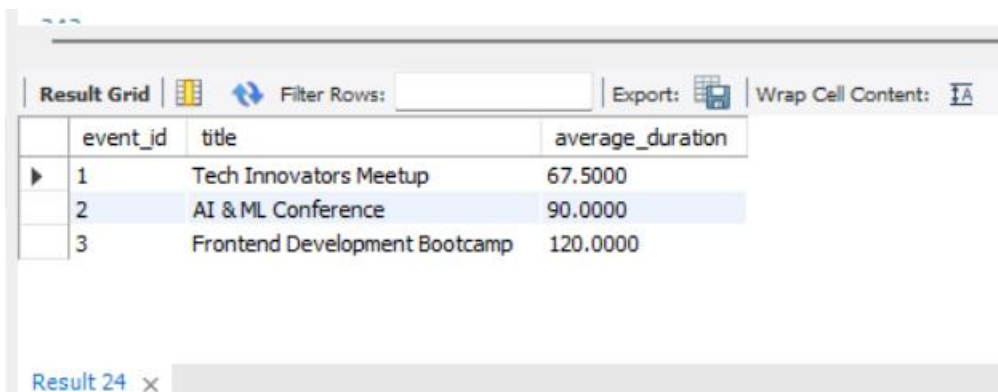
year	month	total_registrations
------	-------	---------------------

Exercise 24: Average Session Duration per Event

Question

Compute the average duration (in minutes) of sessions in each event.

```
SELECT
    e.event_id,
    e.title,
    AVG(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)) AS average_duration
FROM Events e
JOIN Sessions s
ON e.event_id = s.event_id
GROUP BY
    e.event_id,
    e.title;
```



The screenshot shows a database interface with a toolbar at the top containing 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with three columns: 'event_id', 'title', and 'average_duration'. The table contains three rows of data.

event_id	title	average_duration
1	Tech Innovators Meetup	67.5000
2	AI & ML Conference	90.0000
3	Frontend Development Bootcamp	120.0000

Result 24 x

Exercise 25: Events Without Sessions

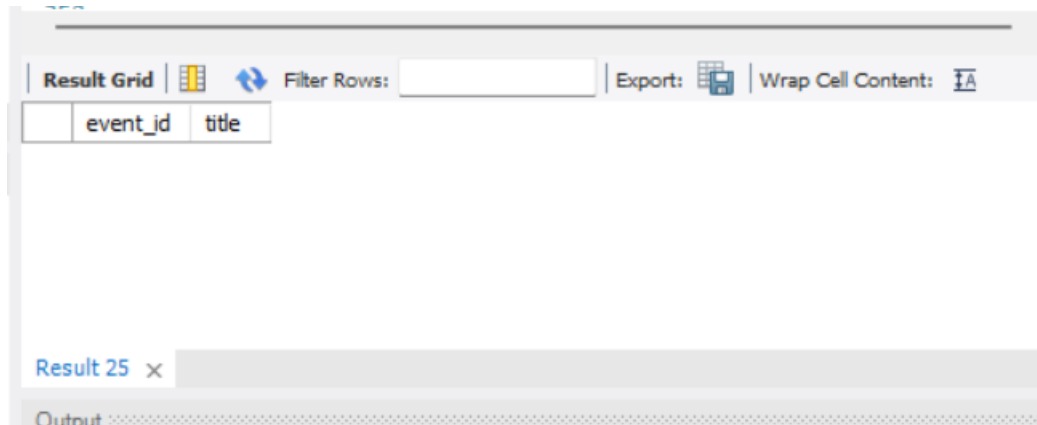
Question

List all events that do not have any sessions scheduled.

```
SELECT
    e.event_id,
```



```
e.title  
FROM Events e  
LEFT JOIN Sessions s  
ON e.event_id = s.event_id  
WHERE s.session_id IS NULL;
```



The screenshot shows a database query result grid. The grid has two columns: 'event_id' and 'title'. The grid is currently empty. The interface includes a toolbar with options like 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The title bar of the window reads 'Result 25 x'.

event_id	title
----------	-------